

Library Management System

Software Requirements Specification (SRS)

Software Engineering - 2

Architecture	Spring Boot Microservices
Cloud & Deployment	Docker + Render
Database	PostgreSQL - separated database per service
Security	JWT + Role-Based Authorization
Version	Improved SRS V2

Table of Contents

1. 1. Introduction
2. 2. Overall Description
3. 3. System Architecture
4. 4. Functional Requirements
5. 5. Non-Functional Requirements
6. 6. System Modules
7. 7. UML Use Case Diagram
8. 8. Activity Diagrams
9. 9. Sequence Diagrams
10. 10. Class Diagram
11. 11. Entity Relationship Diagram (ERD)
12. 12. Object Constraint Language (OCL)
13. 13. Design Patterns Used
14. 14. AOP Usage
15. 15. Docker and Deployment
16. 16. Testing Approach
17. 17. Technical Terms

1. Introduction

1.1 Purpose

The purpose of this project is to develop a complete Library Management System that supports searching, borrowing, reserving, reviewing, and managing books. The system is implemented using Spring Boot REST APIs and Microservices Architecture to make the application modular, scalable, and easier to maintain.

1.2 Scope

The system covers user authentication, book catalog management, borrowing operations, reservations, reviews, favorites, reports, and administration. The backend is divided into independent services and deployed as Dockerized applications on Render.

1.3 Intended Audience

This document is intended for developers, supervisors, testers, system administrators, librarians, and end users.

2. Overall Description

2.1 Product Perspective

The product is a web-based library system. The frontend communicates with a single API Gateway. The gateway routes requests to internal microservices. Each microservice owns its business logic and database.

2.2 Main Users

The system supports three main roles: Admin, Librarian, and User. Each role has different permissions controlled through JWT authentication and role-based authorization.

2.3 Deployment Perspective

The system is deployed on Render using Docker containers. PostgreSQL is used as the database engine. Each business service uses a separate database inside the same PostgreSQL instance.

3. System Architecture

The system uses a microservices architecture. Infrastructure services include Config Server, Eureka Discovery Server, and API Gateway. Business services include Auth, Catalog, Circulation, Engagement, and Report services.

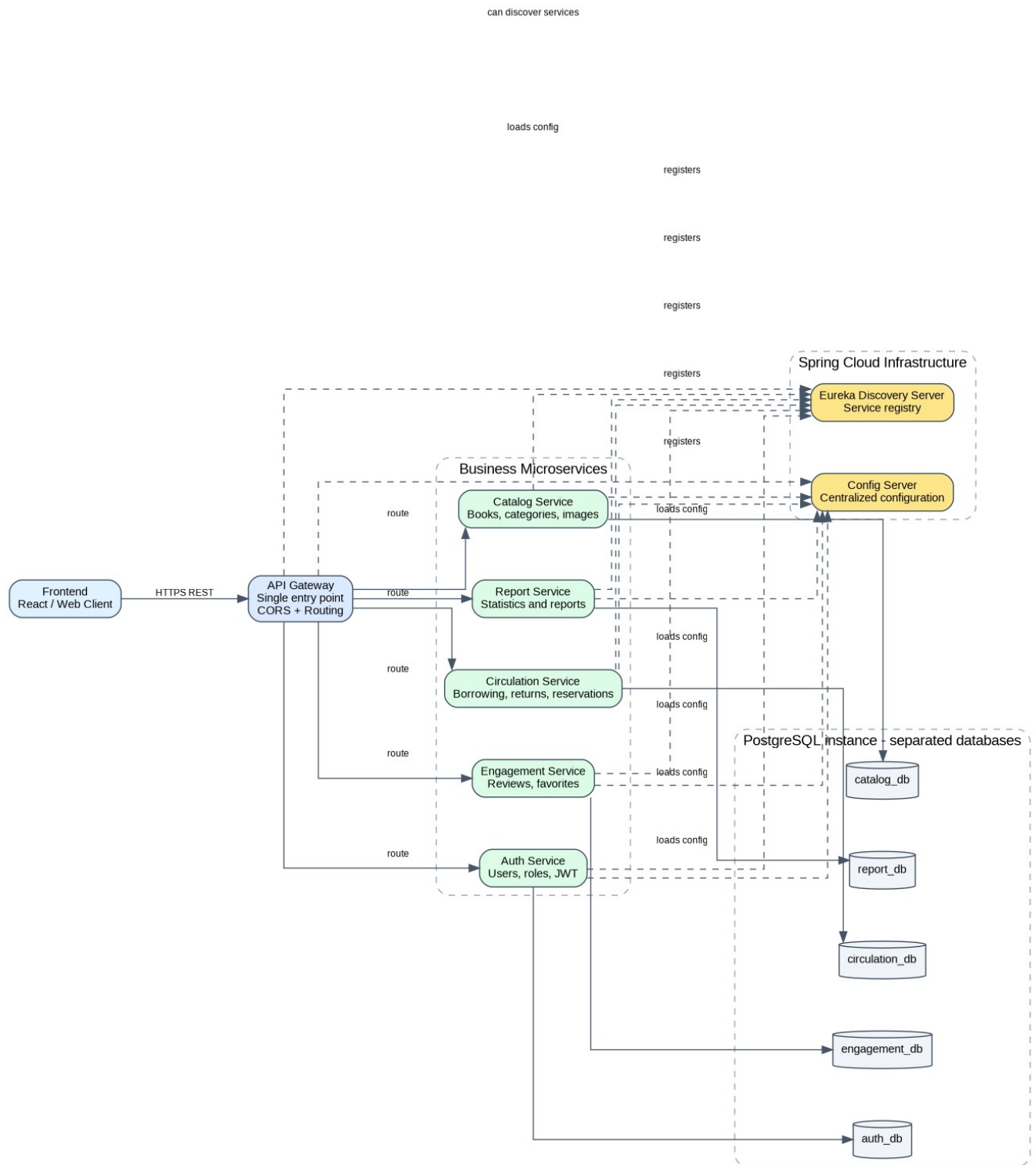


Figure 1: System Architecture Diagram

4. Functional Requirements

ID	Description
FR-1	User can register a new account.

FR-2	User can login securely and receive a JWT token.
FR-3	User can search and view books.
FR-4	User can borrow a book when copies are available.
FR-5	User can reserve a book.
FR-6	User can add, update, and delete reviews.
FR-7	Admin can manage books, categories, users, and reviews.
FR-8	Librarian can approve borrow requests, return books, and manage reservations.
FR-9	Report service can provide system statistics.

5. Non-Functional Requirements

ID	Description
NFR-1 Security	JWT authentication and role-based authorization must protect restricted APIs.
NFR-2 Performance	Book search and normal requests should respond within a reasonable time.
NFR-3 Scalability	Each microservice can be scaled independently.
NFR-4 Maintainability	Layered architecture, DTOs, services, repositories, and AOP make the code easier to maintain.
NFR-5 Availability	Services are Dockerized and deployed on Render to support cloud availability.

6. System Modules

Service	Responsibility	Database
Auth Service	Registration, login, JWT generation, roles, users, audit logs.	auth_db
Catalog Service	Books, categories, authors/book metadata, cover images, availability.	catalog_db
Circulation Service	Borrowing, returning, reservations, transaction history.	circulation_db
Engagement Service	Reviews and favorites.	engagement_db
Report Service	Statistics and summary reports.	report_db

7. UML Use Case Diagram

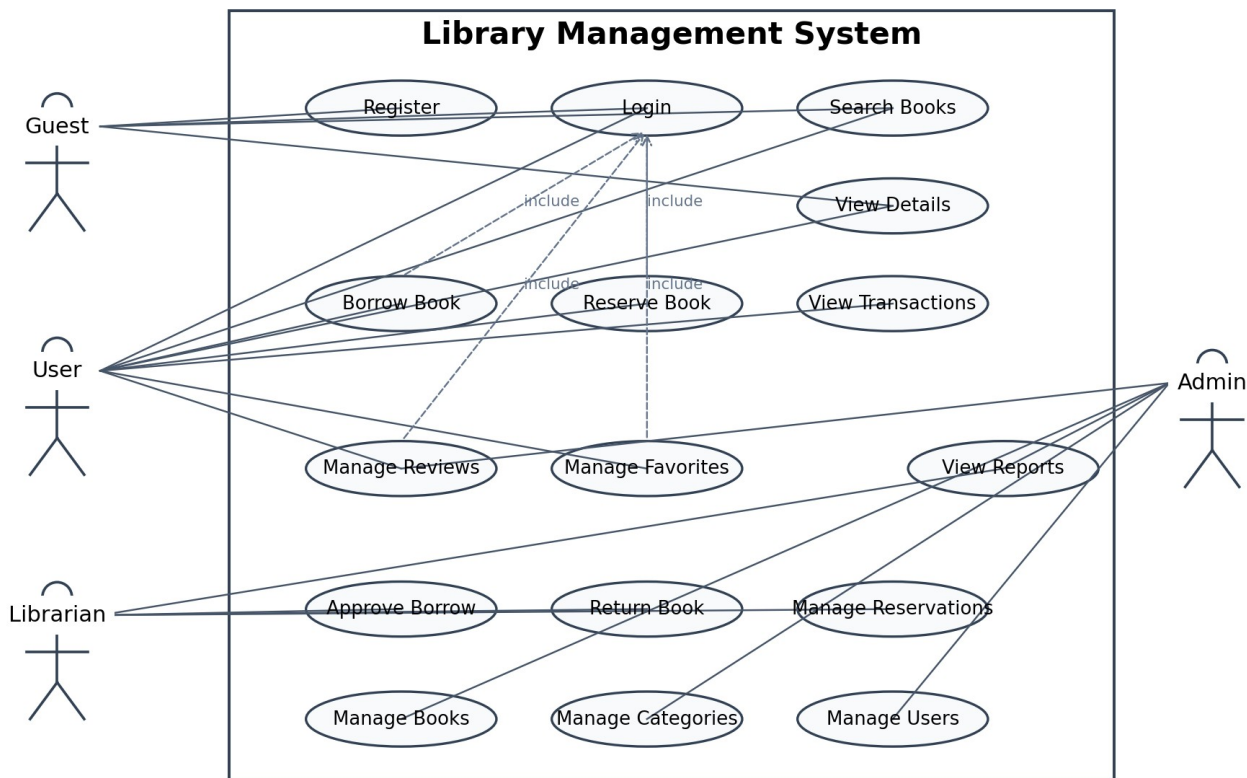


Figure 2: Use Case Diagram

8. Activity Diagrams

8.1 Registration Activity Diagram

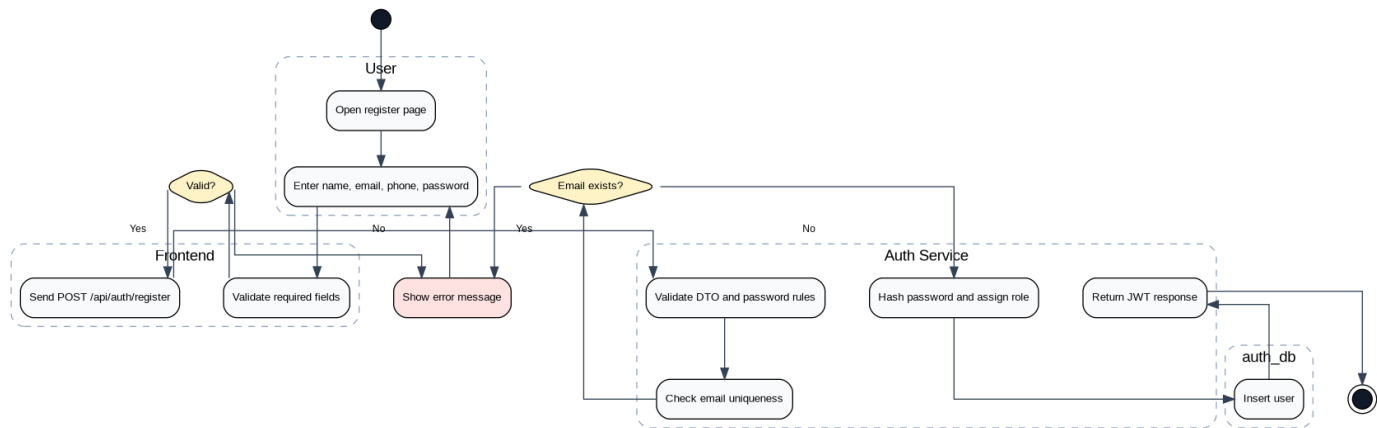


Figure 3: Registration Activity Diagram

8.2 Borrow Book Activity Diagram

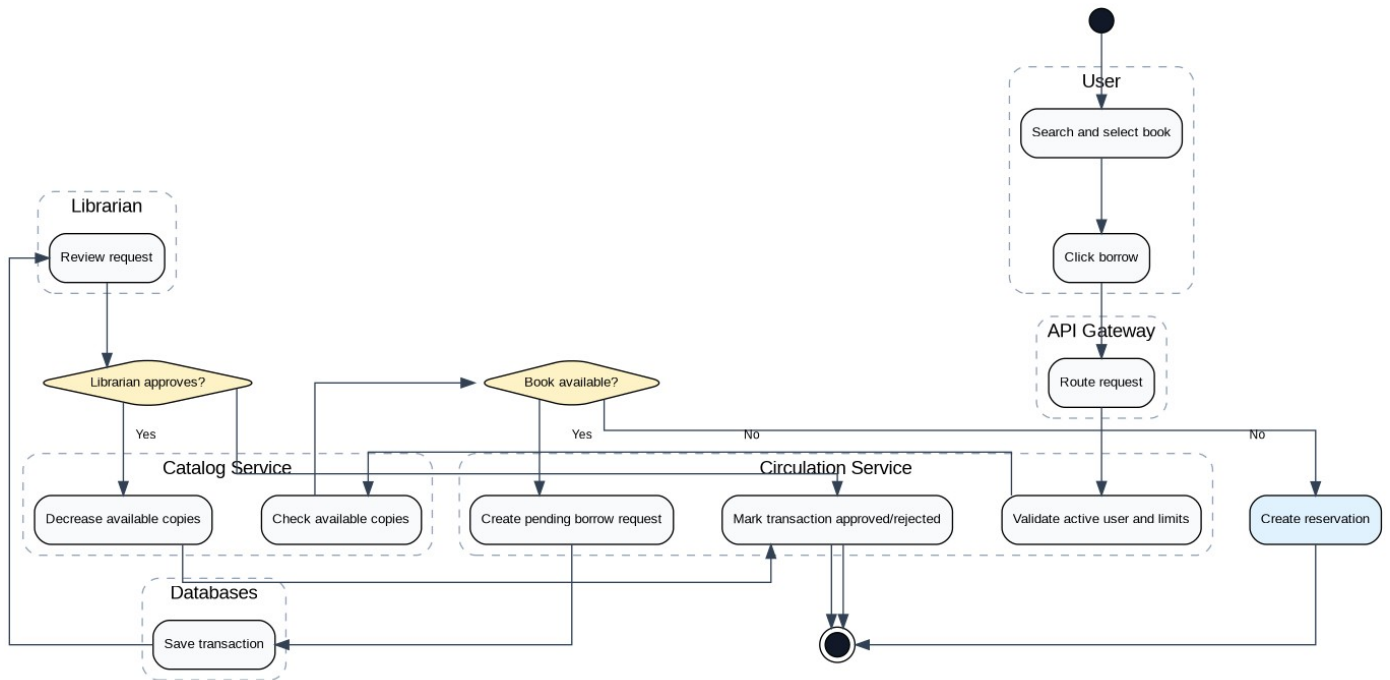


Figure 4: Borrowing Activity Diagram

9. Sequence Diagrams

9.1 Login Sequence Diagram

Sequence Diagram - Login

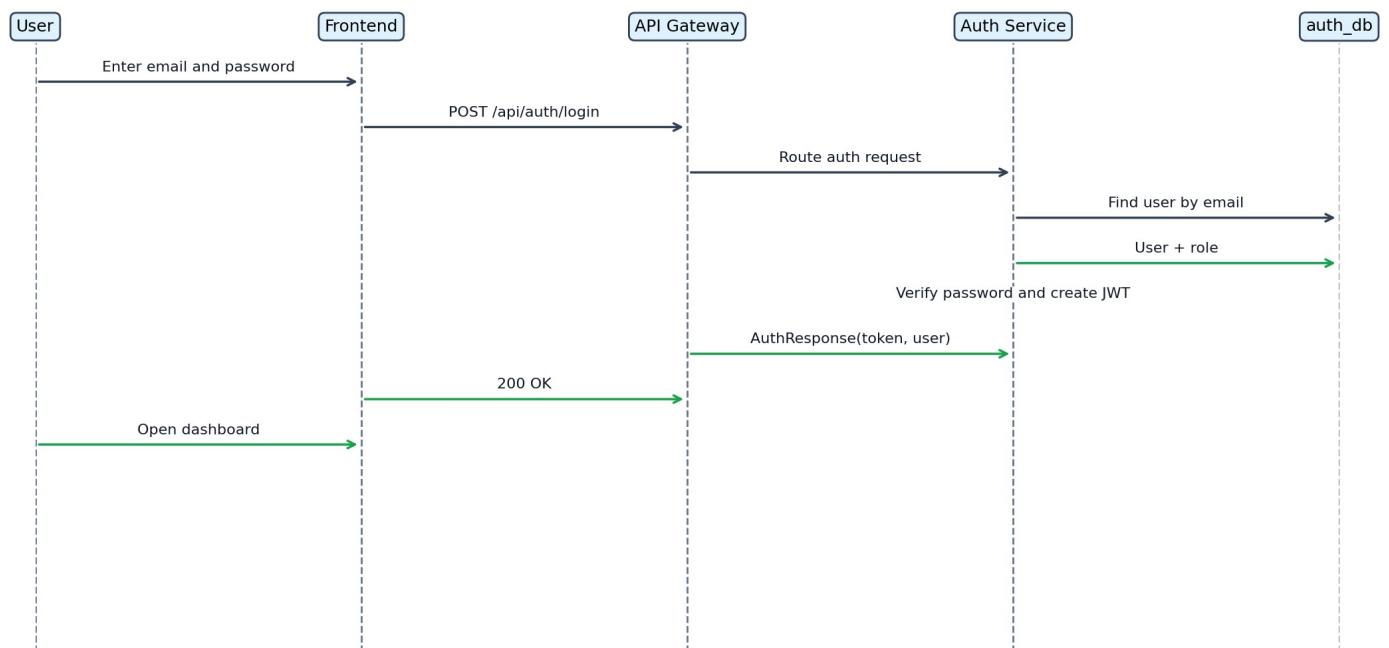


Figure 5: Login Sequence Diagram

9.2 Borrow Request Sequence Diagram

Sequence Diagram - Borrow Request

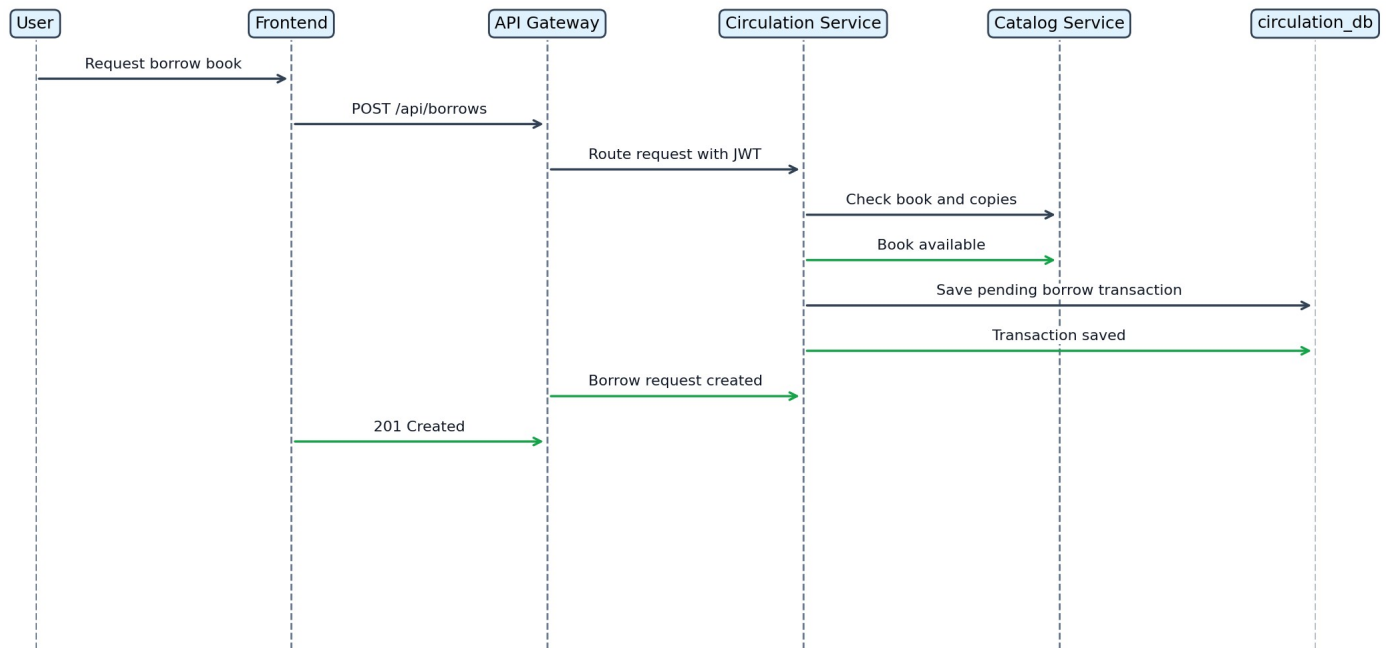


Figure 6: Borrow Request Sequence Diagram

9.3 Add Review Sequence Diagram

Sequence Diagram - Add Review

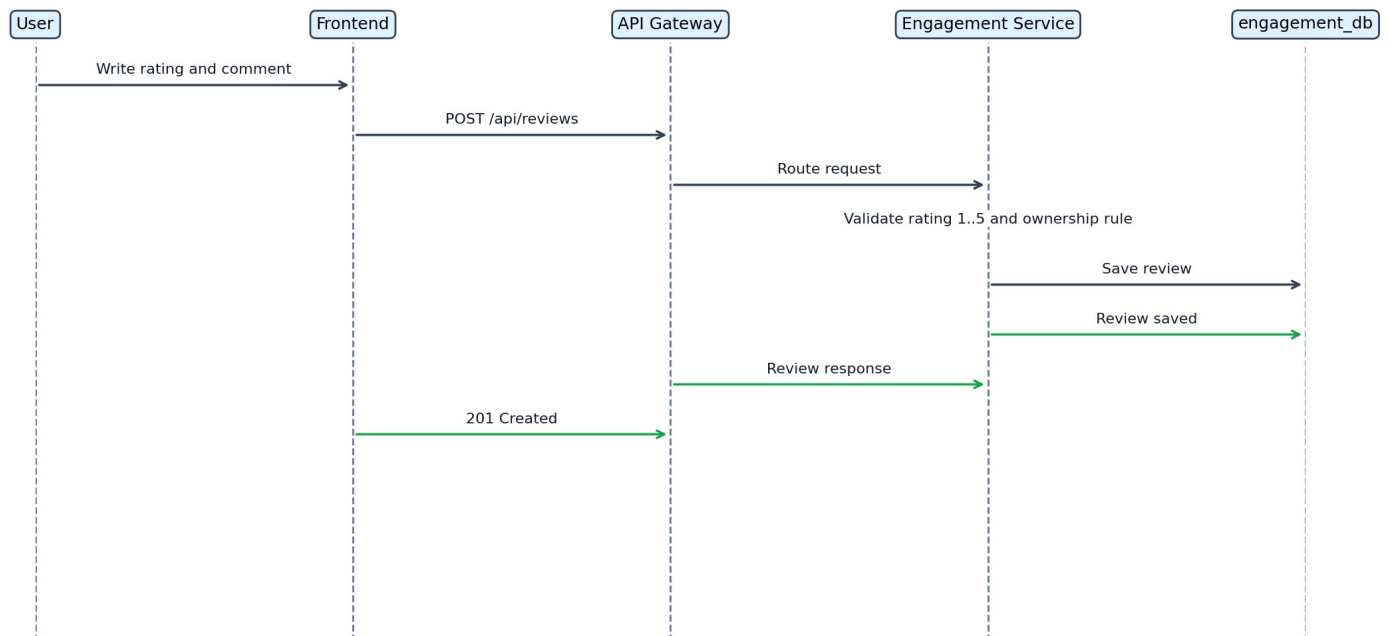


Figure 7: Add Review Sequence Diagram

10. Class Diagram

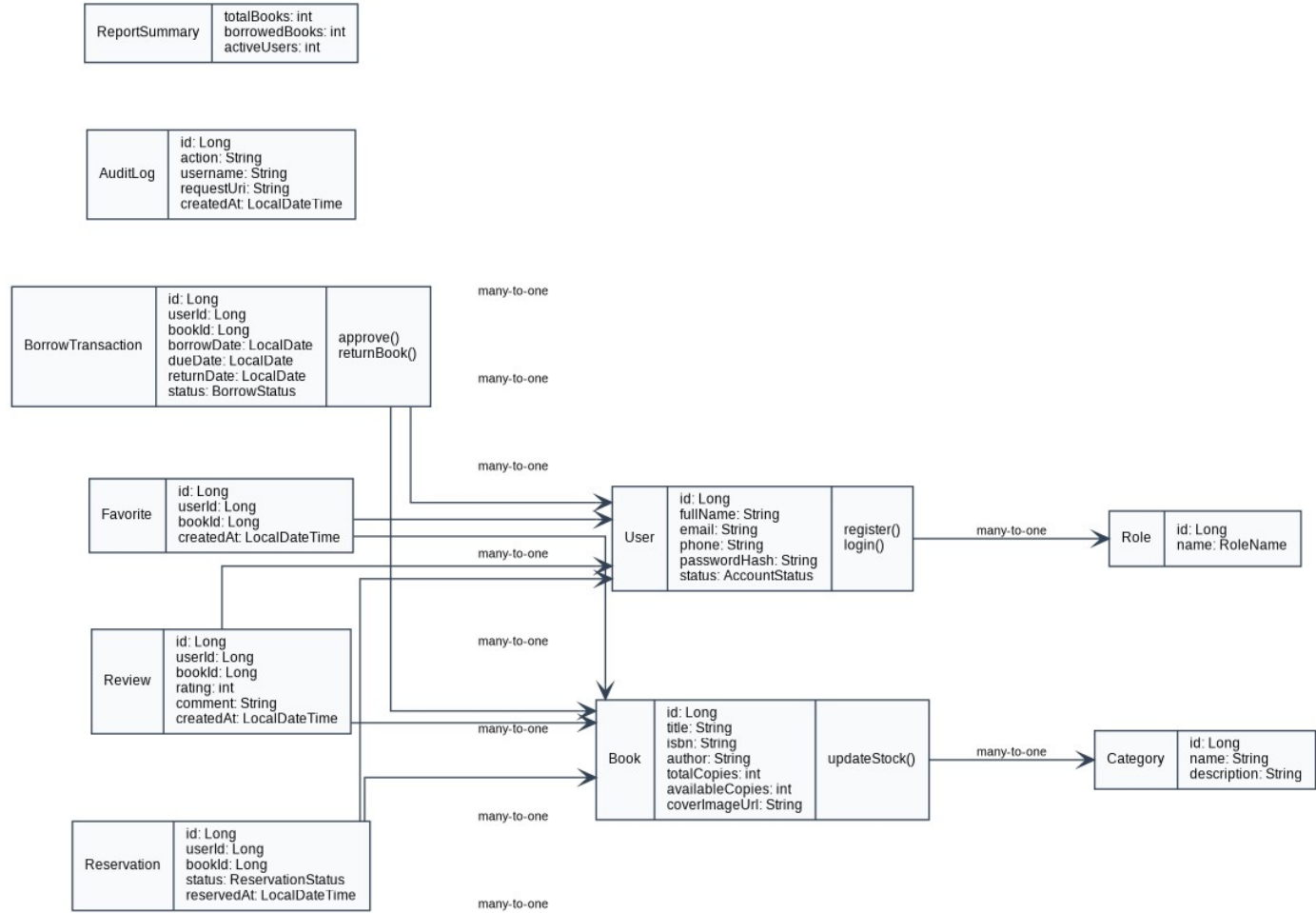


Figure 8: Class Diagram

11. Entity Relationship Diagram (ERD)

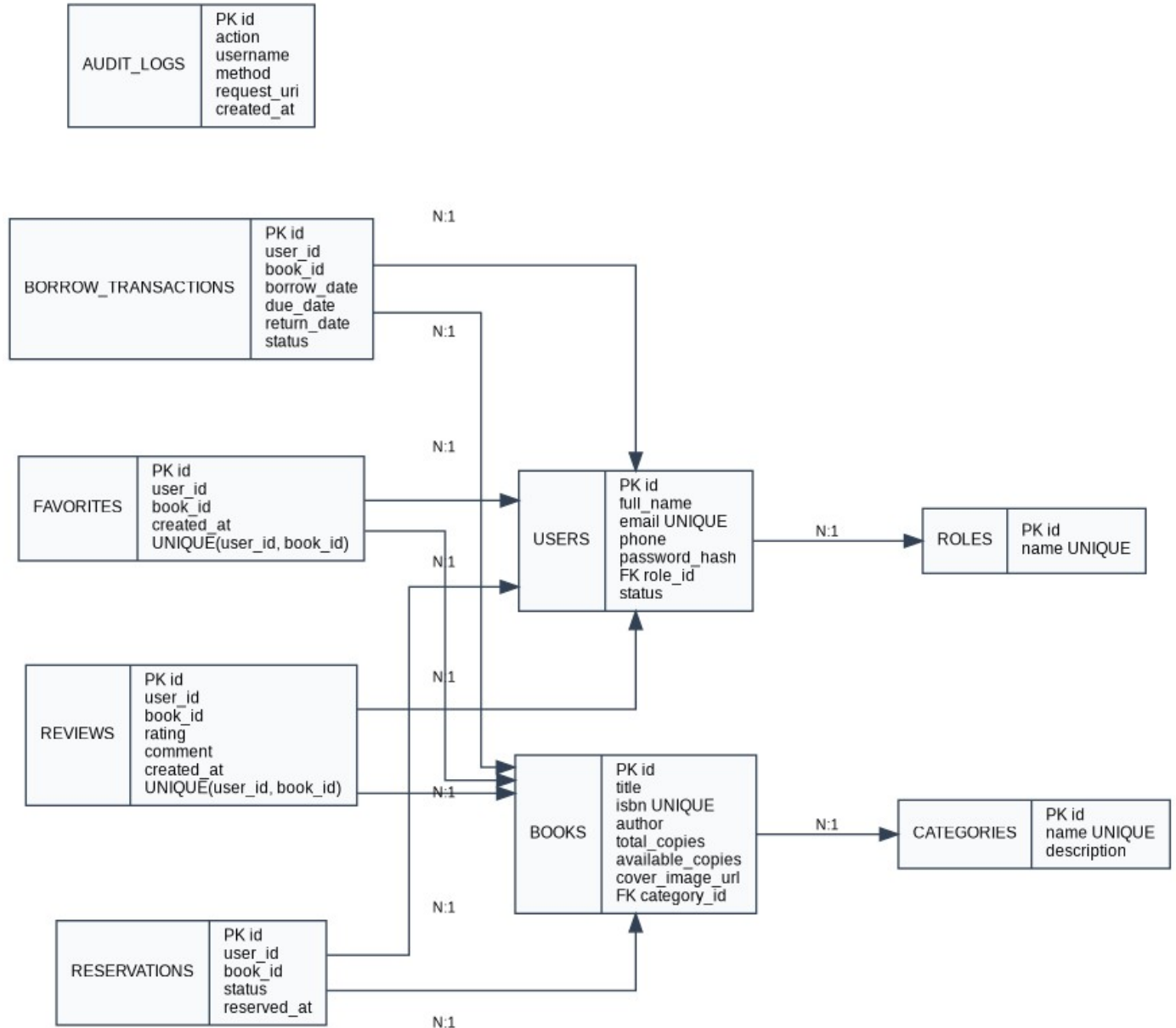


Figure 9: Entity Relationship Diagram

12. Object Constraint Language (OCL)

The following constraints represent the business rules used by the system. They are expressed in OCL form and mapped to the code through DTO validation, service-layer validation, entity constraints, unique database constraints, and authorization checks.

OCL-1 Unique Email

```
context User inv UniqueEmail:
User.allInstances()->isUnique(email)
```

Implementation note: Registration rejects duplicated user emails before saving a user.

OCL-2 Strong Password

```
context RegisterRequest inv StrongPassword:  
password.size() >= 8 and password.matches(passwordPolicy)
```

Implementation note: Registration requires a strong password using validation rules.

OCL-3 Active Account Login

```
context User inv ActiveUserCanLogin:  
self.status = AccountStatus.ACTIVE
```

Implementation note: Inactive users must not access protected APIs.

OCL-4 Valid Book Copies

```
context Book inv ValidCopies:  
self.totalCopies >= 0 and self.availableCopies >= 0 and self.availableCopies <=  
self.totalCopies
```

Implementation note: Catalog service prevents invalid stock values.

OCL-5 Borrow Only If Available

```
context BorrowTransaction inv BorrowRequiresCopy:  
self.book.availableCopies > 0
```

Implementation note: Borrowing can proceed only when a copy is available.

OCL-6 Borrow Limit

```
context User inv BorrowLimit:  
self.activeBorrowTransactions->size() <= 5
```

Implementation note: Circulation service checks borrowing limits before creating a transaction.

OCL-7 Valid Review Rating

```
context Review inv RatingRange:  
self.rating >= 1 and self.rating <= 5
```

Implementation note: Engagement service validates rating range.

OCL-8 One Review Per Book Per User

```
context Review inv UniqueUserBookReview:  
Review.allInstances()->isUnique(r | Tuple{u=r.userId, b=r.bookId})
```

Implementation note: Database and service layer prevent duplicated reviews for the same book by the same user.

OCL-9 One Favorite Per Book Per User

```
context Favorite inv UniqueFavorite:  
Favorite.allInstances()->isUnique(f | Tuple{u=f.userId, b=f.bookId})
```

Implementation note: Engagement service prevents duplicated favorites.

OCL-10 Role-Based Access

```
context ApiRequest inv AdminOnlyManageUsers:  
self.path.startsWith("/api/users") implies self.user.role = ADMIN
```

Implementation note: Security configuration and controllers enforce role-based access.

13. Design Patterns Used

The project uses several common enterprise patterns. These patterns are visible in the structure of controllers, services, repositories, DTOs, Spring components, and Spring Cloud infrastructure.

Pattern	Where Used	Example
Layered Architecture / MVC	Controllers receive HTTP requests, services contain business logic, repositories access the database.	AuthController -> AuthService -> UserRepository
Repository Pattern	Database access is abstracted using Spring Data repositories.	UserRepository, BookRepository, ReviewRepository
DTO Pattern	Request and response objects isolate API contracts from entity models.	LoginRequest, RegisterRequest, AuthResponse, BookRequest, BookResponse
Dependency Injection	Spring injects required beans through constructors instead of manual object creation.	Controller depends on service interface/bean through constructor injection.
Specification Pattern	Dynamic search/filter logic is separated from the repository layer.	BookSpecifications used for book searching and filtering.
API Gateway Pattern	A single gateway receives frontend requests and routes them to backend services.	api-gateway routes /api/auth/**, /api/books/**, etc.
Service Discovery Pattern	Services register themselves with Eureka and can be discovered by service name.	discovery-server with Eureka clients.
Externalized Configuration Pattern	Configuration is separated from code and centralized through Config Server.	config-server and config-repo/*.yaml
Proxy Pattern through AOP	Spring AOP wraps methods to add logging or monitoring without changing business code.	LoggingAspect around service/controller methods.
Builder/Mapper Style DTO Conversion	Responses are assembled from entities into clean API responses.	Service layer converts entities into response DTOs.

14. AOP Usage

AOP is used for cross-cutting concerns such as request logging, method execution logging, error tracing, and performance monitoring. This keeps the main business logic clean and avoids repeating logging code inside every controller or service.

15. Docker and Deployment

Every service is Dockerized and deployed independently on Render. The frontend communicates with the API Gateway. PostgreSQL is hosted as one Render PostgreSQL instance containing separate databases for each business service.

16. Testing Approach

The system can be tested using manual testing, Postman API testing, smoke testing, unit testing with JUnit, black-box testing, and performance testing. Login, registration, book search, borrowing, reviews, and reports are the main tested workflows.

17. Technical Terms

Term	Meaning
Microservice	Independent backend service responsible for one business capability.
API Gateway	Single entry point that routes frontend requests to backend services.
Eureka	Service discovery server used by Spring Cloud.
JWT	Token used to authenticate API requests.
RBAC	Role-Based Access Control.
AOP	Aspect-Oriented Programming used for cross-cutting concerns.
Docker	Containerization tool used to package and deploy services.
Flyway	Database migration tool.
DTO	Object used to transfer request/response data.

Conclusion

The Library Management System satisfies the Software Engineering-2 requirements by implementing REST APIs, microservices, Spring Cloud, Eureka, Config Server, API Gateway, Docker, PostgreSQL, JWT authentication, AOP, OCL constraints, design patterns, and complete SRS diagrams.